

DevOps with AI

Advanced Certification Program

Curriculum





Index

Module 1: Core Concepts (DevOps Certification)

- Devops Fundamentals
- GitOps: Git, GitHub, GitLab
- Infrastructure as Code (IaC): Terraform, AWS CloudFormation
- Microservices: Kubernetes, Docker
- DevSecOps: Jenkins (with security plugins), GitLab CI/CD (with security pipelines)

Module 2: CI/CD Tools

- Jenkins: Job creation, pipeline as code, distributed builds (master-slave architecture)
- GitLab CI/CD: CI/CD pipelines, multi-project pipelines
- GitHub Actions: Advanced workflows, automation tasks

Module 3: Containerisation with Docker & Kubernetes (Docker & Kubernetes Certification)

- Docker: DockerHub, Docker networking, Docker Swarm
- Kubernetes: Pods, Services, Deployments, StatefulSets, ConfigMaps, Secrets
- Istio: Service mesh for Kubernetes, traffic management, security with mTLS
- Tekton: Building CI/CD pipelines in Kubernetes
- FluxCD and Argo CD: GitOps tools for continuous delivery

 Index**Module 4: Cloud & GCP (DevOps Cloud Certification)**

- Cloud-Native Development: Twelve-Factor App methodology, serverless architectures
- AWS Services: Various computing, storage, networking, and database services
- Azure Services: Virtual machines, Azure Kubernetes Service (AKS), Azure Functions
- Continuous Delivery: Canary releases, blue/green deployments
- Security: Zero trust model, mTLS for service-to-service communication

Module 5: Infrastructure as Code (IaC) (Terraform Certification)

- Ansible : Basic and Advanced
- Terraform: Providers, resources, variables, output blocks, dynamic blocks, modules, workspaces, remote state management
- Terraform Cloud: Collaboration, scalability, governance features
- AWS and Azure: Integration with Terraform, infrastructure provisioning

Module 6: Monitoring, Logging, and Observability

- Monitoring: Prometheus, Grafana, Nagios XI
- Logging: ELK Stack (Elasticsearch, Logstash, Kibana)



Index

- Advanced Monitoring: Prometheus (advanced features, alerting), Grafana (advanced features), Jaeger (distributed tracing)
- Alerting and Dashboards: Prometheus Alertmanager, Grafana dashboards
- Scaling Monitoring Infrastructure: Strategies for Prometheus and Grafana

Module 7: DevOps with AI (DevOps with AI Certification)

- AI-Driven Automation
- Predictive Analytics for DevOps
- Incident Management and Root Cause Analysis
- Enhancing Security with AI in DevOp(AIOps)
- AI-Enhanced Monitoring and Observability
- AI-Driven Optimization and Self-Healing Systems



Additional Tools and Technologies Covered

- Version Control: Git, GitHub, GitLab
- CI/CD: Jenkins, GitLab CI/CD, GitHub Actions
- Containerization: Docker
- Container Orchestration: Kubernetes
- Service Mesh: Istio
- Infrastructure as Code: Terraform
- Cloud Platforms: AWS, Azure
- Monitoring and Observability: Prometheus, Grafana, ELK Stack (Elasticsearch, Logstash, Kibana), Jaeger
- GitOps Tools: FluxCD, Argo CD
- Serverless Computing: AWS Lambda, Azure Functions
- Security Tools: Various security plugins integrated into CI/CD pipelines

Empowering your success

1 DevOps Core Concepts

Basic:

- **Session 1: Introduction to DevOps**

- Introduction Introduction to Software Development and DevOps
- DevOps Lifecycle and Tools
- Introduction to DevOps on Cloud
- Linux Fundamentals for DevOps

- **Session 2: Introduction to Advanced DevOps Practices**

- Evolution of DevOps: From CI/CD to GitOps
 - Historical timeline and key milestones in the evolution of DevOps.
 - Evolution from CI/CD practices to the emergence of GitOps.
 - Impact of DevOps on software development lifecycle and operations.
- Historical Overview of DevOps
- Benefits and challenges of GitOps
 - Definition and core principles of GitOps (declarative infrastructure, version control as the single source of truth).
 - Benefits of GitOps: efficiency, traceability, repeatability, and scalability.
 - Challenges of GitOps adoption: cultural shift, tooling complexity, and security considerations.

Case Studies:

- Examples of organizations (e.g., Netflix, Spotify) leveraging GitOps for operational excellence.

1 DevOps Core Concepts

• Session 3: Understanding Infrastructure as Code (IaC)

- Definition and importance of IaC
 - Principles of IaC: automation, consistency, versioning, and scalability.
 - Benefits of IaC in modern IT environments: agility, reproducibility, and cost-effectiveness.
- Common tools and frameworks (Terraform, CloudFormation)
 - Overview and comparison of Terraform and AWS CloudFormation.
 - Practical demonstrations of using Terraform to provision infrastructure.

Intermediate:

• Session 4: Principles of Microservices Architecture

- Introduction to Microservices Architecture
 - Principles and characteristics: modularity, bounded contexts, and decentralized data management.
 - Scalability, resilience, and fault isolation benefits of microservices.
- How microservices influence DevOps practices
 - CI/CD pipelines for microservices: independent deployments and continuous integration challenges.
 - Service discovery, load balancing, and monitoring considerations.
- Comparing monolithic and microservice architecture
 - Contrasting characteristics: deployment complexity, scaling limitations, and technology stack flexibility.
 - Case studies of successful microservices migrations.

1 DevOps Core Concepts

• Session 5: Introduction to DevSecOps

- Principles of DevSecOps
 - Shift-left security: integrating security throughout the software development lifecycle (SDLC).
 - Security automation: vulnerability scanning, static code analysis, and continuous monitoring.
- Comparison of DevSecOps tools with traditional DevOps tools
 - Traditional DevOps vs. DevSecOps tooling: Jenkins vs. Jenkins with security plugins, GitLab CI/CD security pipelines.
 - Container security tools (e.g., Docker Bench, Clair) and infrastructure security as code practices.
- DevSecops
 - High-profile security breaches and lessons learned.
 - Implementing DevSecOps in regulatory environments (e.g., PCI-DSS compliance).

• Session 6: Basic Git Workflow

- Introduction to Microservices Architecture
 - Principles and characteristics: modularity, bounded contexts, and decentralized data management.
 - Scalability, resilience, and fault isolation benefits of microservices.
- How microservices influence DevOps practices
 - CI/CD pipelines for microservices: independent deployments and continuous integration challenges.
 - Service discovery, load balancing, and monitoring considerations.
- Comparing monolithic and microservice architecture
 - Contrasting characteristics: deployment complexity, scaling limitations, and technology stack flexibility.
 - Case studies of successful microservices migrations.

1 DevOps Core Concepts

Advanced:

- **Session 7: Advanced Git Usage**

- Git internals and advanced branching strategies
 - Understanding Git objects: blobs, trees, commits, and tags.
 - Exploring the Git repository structure and object hashing.
- GitFlow, GitHub Flow, GitLab Flow
 - GitFlow: feature branches, release branches, and hotfix branches.
 - GitHub Flow: continuous deployment with mainline branches.
 - GitLab Flow: leveraging CI/CD pipelines with version control.
 - Rebase vs. merge strategies: interactive rebase and squash commits.
 - Git hooks: automating pre-commit and post-commit actions.

- **Session 8: Git Workflow Patterns for Collaboration and Release Management**

- Release Branching and Release Tagging Strategies
 - Semantic versioning: tagging releases and managing changelogs.
 - Release branching models: trunk-based development vs. feature branches.
- Git Workflow Automation with GitHub Actions and GitLab CI/CD
 - Integrating Git with CI/CD pipelines: automating builds, tests, and deployments.
 - Leveraging GitHub Actions and GitLab CI/CD for workflow automation.
 - Declarative configuration management with GitOps principles.
 - Continuous delivery with infrastructure as code (IaC) and version-controlled environments.

1 DevOps Core Concepts

- **Session 9: Case Studies and Practical Labs**

- Real-world case studies
 - Hands-on lab: configuring CI/CD pipelines for Git-based workflows.
 - Hands-on lab: Applying Git, IaC, and CI/CD principles to solve practical challenges.
- Practical exercises
 - Analyzing case studies of successful GitOps implementations in enterprise environments.



2

CI/CD Tools

Basic:

- **Session 10: Mastering Jenkins**
- Job creation, plugin management
 - Overview of Jenkins as a CI/CD tool.
 - Installation, setup, and basic configuration.
 - Creating and configuring Jenkins jobs: freestyle vs. pipeline jobs.
 - Best practices for job naming conventions, parameterization, and job dependencies.
 - Exploring essential Jenkins plugins for various functionalities (e.g., Git, Docker, SonarQube).
 - Managing plugin updates, installation, and compatibility.
 - Configuring authentication and authorization in Jenkins.
 - Implementing security best practices for Jenkins instances.

Hands-on Lab:

- Setting up a Jenkins server.
- Creating and configuring Jenkins jobs with plugins for automated testing and deployment.

Intermediate:**• Session 11: Jenkins Pipeline as Code**

- Declarative pipeline syntax
 - Introduction to Jenkinsfile: structure, stages, and steps.
 - Writing declarative pipelines for building, testing, and deploying applications.
- Integrating security tools into CI/CD pipelines
 - Integrating static code analysis tools (e.g., SonarQube, Checkmarx) into CI/CD pipelines.
 - Implementing security gates and automated vulnerability assessments.
 - Version controlling Jenkinsfile: best practices for managing pipeline code.
 - Automated testing and validation of pipeline changes.

Hands-on Lab:

- Creating a Jenkins Pipeline using declarative syntax.
- Integrating security checks into the pipeline and configuring automated tests.

Advanced:**• Session 12: Advanced Jenkins Configurations**

- Distributed Builds and Scalability with Jenkins Master-Slave Architecture
 - Setting up Jenkins master-slave architecture for parallel builds.
 - Load balancing and scaling Jenkins agents for distributed environments.

2

CI/CD Tools

- Advanced Pipeline Configuration: Variables, caching, and retries
 - Using environment variables, caching dependencies, and retry mechanisms in pipelines.
 - Handling failures gracefully and implementing retry strategies.
 - Monitoring and optimizing Jenkins performance: memory allocation, disk space management.
 - Configuring Jenkins for high availability and disaster recovery.

- **Hands-on Lab:**

- Configuring Jenkins master-slave setup.
 - Implementing advanced pipeline features like caching and retry mechanisms.

- **Session 13: Multi-project Pipelines and Cross-project Dependencies**

- GitLab CI/CD Pipelines
 - Orchestrating pipelines across multiple repositories and projects.
 - Managing dependencies and triggering builds across interconnected projects.
 - Introduction to GitLab CI/CD: CI/CD configuration in GitLab CI/CD YAML.
 - Leveraging GitLab CI/CD for multi-project pipelines and integration with GitLab features (e.g., Merge Requests).
- GitHub Actions: Advanced workflows
 - Advanced workflows in GitHub Actions: matrix builds, conditional steps, and environment deployments.

2

CI/CD Tools

- Advanced workflows in GitHub Actions: matrix builds, conditional steps, and environment deployments.
- Integrating GitHub Actions with GitHub features (e.g., Pull Requests, Code Reviews).

- **Hands-on Lab:**

- Implementing multi-project pipelines using GitLab CI/CD.
- Setting up advanced workflows and automation tasks in GitHub Actions.

Practical Labs:

- **Session 14–16: Hands-on Labs**

- Practical labs for Jenkins, GitLab CI/CD, GitHub Actions
 - Rotating labs covering Jenkins basics, Pipeline as Code, advanced configurations, and multi-project pipelines.
 - Practical exercises aligning with industry-standard use cases and deployment scenarios.
 - Simulated deployment scenarios: continuous integration, automated testing, and deployment to different environments.
 - Troubleshooting common issues in CI/CD pipelines and optimizing pipeline performance.
 - Peer review sessions: sharing insights and optimizing CI/CD pipelines based on feedback.
 - Iterative development: refining pipeline scripts and configurations to meet evolving project requirements.

3

Containerization with Docker & Kubernetes

Basic:

- **Session 17: Introduction to Docker**

- Docker Images and DockerHub
 - Overview of containerization: benefits and use cases in software development and deployment.
 - Introduction to Docker architecture: Docker daemon, Docker client, and Docker registry (DockerHub).
- Dockerfile and building custom images
 - Creating Docker images: Dockerfile syntax, layers, and best practices.
 - Publishing and managing Docker images on DockerHub.
- Docker Port Mapping
 - Writing Dockerfiles for building custom Docker images.
 - Best practices for optimizing Docker images: minimizing image size and improving build efficiency.
 - Basics of container networking: Docker network drivers, bridge networks, and host networking.
 - Use cases for container communication and network isolation.

Intermediate:

- **Session 18: Docker Networking**

- Container Networking Interfaces (CNIs)
 - Overview of CNIs and their role in container networking.
 - Implementing custom network configurations using CNIs.
 - Configuring overlay networks for multi-host communication in Docker Swarm and Kubernetes.
 - Security considerations and network policies in containerized environments.

3

Containerization with Docker & Kubernetes

- Using Docker Swarm and Kubernetes for service discovery and load balancing.
- Deploying applications with multiple containers and managing inter-container communication.

• Session 19: Docker Swarm Mode

- Introduction and basic configurations
 - Overview of Docker Swarm architecture: manager nodes, worker nodes, and service orchestration.
 - Deploying services in Docker Swarm: stack deployment, service scaling, and rolling updates.
 - Configuring overlay networks in Docker Swarm for multi-host communication.
 - Load balancing and service discovery with Swarm mode.
 - Implementing high availability strategies in Docker Swarm: manager node redundancy and fault tolerance.
 - Handling node failures and ensuring service continuity.

• Session 20: Kubernetes Components and Architecture

- Overview of Kubernetes architecture
 - Master and worker nodes: kube-apiserver, kube-controller-manager, kube-scheduler, etcd, and kubelet.
 - Understanding Kubernetes cluster architecture and control plane components.
- Kubernetes Pods and multi-container Pods
 - Anatomy of a Pod: containers, shared storage volumes, and networking.
 - Deploying multi-container Pods and managing inter-container communication.
 - Strategies for deploying applications with Kubernetes Deployments.
 - Rolling updates, rollback strategies, and managing application versions.

3 Containerization with Docker & Kubernetes

• Session 21: Kubernetes Services and Deployments

- Services: Types and configurations
 - Types of Kubernetes Services: ClusterIP, NodePort, LoadBalancer, and ExternalName.
 - Service discovery and load balancing within a Kubernetes cluster.
- Deployments: Strategies and use cases
 - Using Kubernetes Deployments for declarative application management.
 - Strategies for zero-downtime deployments, blue-green deployments, and canary releases.
 - ConfigMaps and Secrets: managing application configurations and sensitive data in Kubernetes.
 - Using Helm for managing Kubernetes application packages and templating.

Advanced:

• Session 22: Kubernetes Storage and Persistent Volume

- Stateful sets, Config Maps, and Secrets
 - Deploying stateful applications with Kubernetes StatefulSets.
 - Managing pod identity, persistent storage, and ordered deployment and scaling.
 - Overview of Kubernetes Persistent Volumes: storage classes, volume plugins, and dynamic provisioning.
 - Configuring and managing PVs and Persistent Volume Claims (PVCs).
 - Managing application configurations and sensitive data using ConfigMaps and Secrets.
 - Best practices for securing and accessing secrets in Kubernetes environments.

3

Containerization with Docker & Kubernetes

• Session 23: Advanced Kubernetes Scheduling Techniques

- Affinity, Anti-affinity, and Taints/Tolerations
 - Using node affinity and pod affinity/anti-affinity to influence pod placement in Kubernetes.
 - Strategies for optimizing application performance and availability using affinity rules.
- Resource Management: Resource Quotas, Limit Ranges
 - Kubernetes resource quotas: defining resource limits and usage constraints for namespaces.
 - Limit ranges: configuring default resource requests and limits for Kubernetes pods.

• Session 24: Kubernetes in Production

- Advanced deployments: Canary, Blue/Green
 - Canary deployments: gradually rolling out new versions and gathering user feedback.
 - Blue-green deployments: switching traffic between multiple identical environments.
- Autoscaling: Horizontal Pod Autoscaler, Cluster Autoscaler, Vertical Pod Autoscaler
- Horizontal Pod Autoscaler (HPA): automatic scaling based on CPU or custom metrics.
- Cluster Autoscaler: scaling Kubernetes clusters based on resource usage and pending pods.
- Vertical Pod Autoscaler: optimizing resource allocation for individual pods based on resource usage patterns.
- Implementing monitoring solutions (e.g., Prometheus, Grafana) for Kubernetes clusters.
- Configuring logging with Elasticsearch, Fluentd, and Kibana (EFK stack) for centralized log management.

3

Containerization with Docker & Kubernetes

- **Session 25: Kubernetes with Istio**

- Traffic Management
 - Overview of Istio architecture: Envoy proxy, Mixer, Pilot, and Istio control plane.
 - Installing and configuring Istio in a Kubernetes cluster.
- Request Routing, Traffic Shifting, and Load Balancing
 - Implementing request routing, traffic shifting, and load balancing with Istio VirtualServices.
 - Canary deployments and A/B testing using Istio traffic management features.
 - Securing service-to-service communication with Istio mTLS (mutual TLS) authentication.
 - Distributed tracing with Jaeger for monitoring and troubleshooting microservices interactions.

- **Session 26: Tekton and GitOps**

- Building CI/CD Pipelines with Tekton
 - Introduction to Tekton: architecture, Tasks, Pipelines, and PipelineRuns.
 - Building and managing CI/CD pipelines with Tekton in Kubernetes clusters.
- Implementing GitOps with FluxCD and Argo CD
 - GitOps principles and practices: declarative infrastructure and continuous delivery.
 - Setting up and configuring FluxCD for automated GitOps workflows.
 - Managing application deployments with Argo CD and GitOps best practices.

3

Containerization with Docker & Kubernetes

Practical Labs:

- **Session 27–28: Hands-on Labs**

- Practical labs for GitOps, Docker and Kubernetes
 - Rotating labs covering Docker basics, Kubernetes networking, storage, advanced scheduling, Istio traffic management, and GitOps principles.
 - Practical exercises aligning with industry-standard use cases and deployment scenarios.
- Scenario-based Exercises:
 - Simulated deployment scenarios: multi-container applications, stateful applications with persistent storage, and advanced deployment strategies.
 - Troubleshooting common issues in Kubernetes deployments and optimizing application performance.

Empowering your success

4

Cloud & GCP

Basic:

• **Session 29: Cloud-Native Development Patterns**

- Twelve-Factor App Methodology and Cloud-Native Design Patterns
 - Principles of twelve-factor apps: best practices for building scalable, maintainable applications.
 - Understanding each factor: environment parity, dependencies management, configuration, and backing services.
 - Design patterns for microservices and serverless architectures.
 - Patterns for resilience, scalability, and observability in cloud-native applications.
 - Analyzing benefits and challenges of adopting cloud-native development approaches.

Intermediate:

• **Session 30: Building Cloud-Native Applications**

- With Kubernetes and Serverless Architectures
 - Container orchestration with Kubernetes: deployment strategies and scaling applications.
 - Using Kubernetes for managing microservices and distributed applications.
 - Introduction to serverless computing: benefits and limitations.
 - Implementing serverless applications on cloud platforms (e.g., AWS Lambda, Google Cloud Functions).
 - Integrating Kubernetes and serverless components in hybrid cloud environments.
 - Choosing between Kubernetes and serverless based on application requirements and use cases.

- **Session 31: Implementing Continuous Delivery for Cloud-Native Applications**

- Progressive Delivery Techniques: Canary Releases, Blue/Green Deployments
 - Principles and benefits of continuous integration and continuous deployment (CI/CD) for cloud-native apps.
 - Building automation pipelines for CI/CD in cloud environments.
 - Canary releases: gradual rollout and testing of new features in production.
 - Blue/green deployments: switching traffic between multiple environments for zero-downtime updates. Implementing feature toggles and A/B testing strategies in cloud-native applications.
 - Managing feature flags dynamically and their impact on continuous delivery pipelines.

Intermediate:

- **Session 32: Securing Cloud-Native Applications**

- Zero Trust Security Model for Cloud-Native Environments
 - Principles of zero trust security for cloud-native environments.
 - Implementing identity-based access controls and least privilege principles.
- Secure Service-to-Service Communication with mTLS and SPIFFE/SPIRE
 - Mutual TLS (mTLS) authentication for securing communication between microservices.
 - Using SPIFFE (Secure Production Identity Framework for Everyone) and SPIRE (SPIFFE Runtime Environment) for identity management.

4

Cloud & GCP

- Securing APIs, containers, and orchestrators in cloud environments.
- Monitoring and auditing for compliance and threat detection in cloud-native applications.

Intermediate:**• Session 33: Overview of AWS and Azure**

- Key services and DevOps integrations
 - Overview of core AWS services: compute, storage, networking, databases, and analytics.
 - Integration of AWS services with DevOps tools and practices.
 - Overview of key Azure services: virtual machines, Azure Kubernetes Service (AKS), Azure Functions, and Azure databases.
 - Azure DevOps: CI/CD pipelines, integration with Azure services, and infrastructure as code (IaC) with Azure Resource Manager (ARM).
 - Contrasting AWS and Azure offerings in terms of scalability, pricing, and service-specific features.
 - Choosing the right cloud platform based on application requirements and organizational needs.

Practical Labs:**• Session 34: Hands-on Labs**

- Practical lab for cloud-native development and security
 - Setting up cloud-native applications using Kubernetes and serverless technologies.
 - Implementing CI/CD pipelines for cloud-native applications.
 - Securing microservices communication with mTLS and exploring zero trust security models.

5 Infrastructure as Code (IaC)

Basics:

- **Session 35: Terraform Basics**
 - Providers, Resources, Variables, Output Blocks, Dynamic Blocks
 - Overview of Infrastructure as Code (IaC) principles and benefits.
 - Understanding Terraform's declarative syntax and its advantages.
 - Terraform Configuration Language (HCL)
 - Writing Terraform configuration files: providers, resources, variables, and output blocks.
 - Managing infrastructure components (AWS, Azure, Google Cloud) using Terraform.
 - State Management:
 - Importance of Terraform state: tracking resources and managing changes.
 - State file formats, state locking, and handling concurrent Terraform operations.
 - Dynamic Blocks:
 - Utilizing dynamic blocks in Terraform configurations for flexible resource definitions.
 - Best practices for using dynamic blocks effectively.

Advanced:

- **Session 36: Terraform Modules and Workspaces**
 - Creating reusable modules
 - Terraform Modules:
 - Creating and using Terraform modules for reusable infrastructure components.
 - Structuring modules for scalability and maintainability.

4 Infrastructure as Code (IaC)

- Managing environments with workspaces
 - Overview of Terraform Cloud: benefits for collaboration, remote state management, and version control.
 - Integrating Terraform Cloud with CI/CD pipelines and version control systems.
- Terraform Cloud: Collaboration and Remote State Management

Advanced:

• Session 37: Advanced Terraform Features

- Collaboration features, Remote State Management, and Locking
 - Collaboration Features:
 - Implementing collaboration workflows with Terraform Enterprise or Terraform Cloud.
 - Role-based access control (RBAC), audit logs, and managing Terraform configurations across teams.
 - Remote State Management:
 - Strategies for remote state storage: Terraform Cloud, AWS S3, Azure Blob Storage.
 - Enhancing security and scalability with remote state management.
 - Terraform Enterprise: Scalability and Governance
 - Deploying Terraform Enterprise for large-scale infrastructure management.
 - Governance practices: policy as code, Sentinel policies, and compliance enforcement.

5 Infrastructure as Code (IaC)

Practical Labs:

- **Session 38: Hands-on Lab**

- Practical lab for Terraform
 - Setting up Terraform environment: installing Terraform CLI, configuring providers, and initializing projects.
 - Creating Terraform configurations for provisioning infrastructure resources (e.g., virtual machines, networks, storage).
 - Managing state files, workspaces, and modules in Terraform.
 - Integrating Terraform with CI/CD pipelines for automated infrastructure deployments.
 - Best practices for using dynamic blocks effectively.

5

Monitoring, Logging, and Observability**Basic:**

- **Session 39: Introduction to Monitoring and Logging**
 - Basic Monitoring with Prometheus, Grafana, Nagios XI
 - Basic Monitoring:
 - Overview of monitoring principles: metrics, metrics collection, and visualization.
 - Introduction to Prometheus:
 - Installation and configuration.
 - Defining and querying metrics.
 - Grafana for visualization:
 - Setting up Grafana dashboards.
 - Creating visualizations and charts.
 - Overview of Nagios XI for alerting and monitoring infrastructure health.
 - Basic Logging with ELK Stack (Elasticsearch, Logstash, Kibana)
 - Overview of ELK Stack components:
 - Elasticsearch: indexing and storing logs.
 - Logstash: log aggregation and parsing.
 - Kibana: log search and visualization.
 - Configuring Logstash pipelines for log processing.
 - Creating dashboards and visualizations in Kibana.

Advanced

- **Session 40: Advanced Monitoring and Observability**
 - Advanced Monitoring with Prometheus and Grafana
 - Prometheus advanced features:
 - Service discovery mechanisms.
 - Recording rules and alerts.
 - Advanced Grafana features:
 - Templating and variables.
 - Dashboard annotations and annotations.

7

DevOps with AI

Basic:

• **Session 41: Introduction to AI in DevOps**

- Overview of AI and its relevance to DevOps
- Benefits of integrating AI in DevOps: automation, predictive analytics, and enhanced decision-making
- Key AI techniques and tools for DevOps: machine learning, neural networks, and natural language processing
- Case studies: How leading organizations use AI in DevOps to improve efficiency and reliability

• **Session 42: AI-Driven Automation in DevOps**

- AI in CI/CD pipelines: automating testing, deployment, and monitoring
- Leveraging AI for code quality analysis and bug detection
- Tools and frameworks for AI-driven automation: TensorFlow, PyTorch, and OpenAI tools
- Practical demonstration: Implementing AI-driven automation in a sample CI/CD pipeline

Intermediate:

• **Session 43: Predictive Analytics for DevOps**

- Introduction to predictive analytics and its role in DevOps
- Use cases: performance forecasting, capacity planning, and anomaly detection
- Tools and platforms for predictive analytics in DevOps: DataRobot, Splunk, and ELK Stack with machine learning plugins
- Practical demonstration: Implementing predictive analytics for performance monitoring.

7

DevOps with AI

- **Session 44: AI for Incident Management and Root Cause Analysis**

- AI-driven incident management: reducing MTTR with automated diagnostics
- Using machine learning for root cause analysis: techniques and best practices
- Tools for AI-driven incident management: IBM Watson AIOps, Moogsoft, and BigPanda
- Practical demonstration: Implementing AI-driven incident management and root cause analysis

- **Session 45: Enhancing Security with AI in DevOps (AIOps)**

- Introduction to AIOps: AI for IT operations and security
- Using AI for threat detection, vulnerability management, and compliance
- Tools for AIOps: Darktrace, Sumo Logic, and Splunk with AI integrations
- Practical demonstration: Implementing AI for security monitoring and threat detection

- **Session 46: AI-Enhanced Monitoring and Observability**

- AI for monitoring: anomaly detection, pattern recognition, and predictive alerts
- Integrating AI with existing monitoring tools: Prometheus, Grafana, and ELK Stack
- Case studies: Real-world examples of AI-enhanced monitoring in DevOps
- Practical demonstration: Setting up AI-enhanced monitoring for a sample application.

7

DevOps with AI

Advanced

- **Session 47: AI-Driven Optimization and Self-Healing Systems**

- AI for resource optimization: dynamic scaling and resource allocation
- Implementing self-healing systems with AI: automated remediation and fault tolerance
- Tools for AI-driven optimization: Kubernetes with AI integrations, Google Cloud AI
- Practical demonstration: Building a self-healing system using AI and Kubernetes.

- **Session 48: Practical Labs and Real-World Applications**

- Hands-on lab: Implementing AI-driven CI/CD pipeline
- Hands-on lab: AI for predictive analytics and performance monitoring
- Hands-on lab: AI-driven incident management and root cause analysis
- Analyzing case studies of successful AI integration in DevOps.



www.skillfyme.in